EST-744

Transformers y Modelos Pre-entrenados

Juan Zamora O.

Deep Learning





Anteriormente

Clase anterior: Mecanismo de atención como componente aislado.

Esta clase: Arquitectura Transformer completa.

De la Atención al Transformer

El Transformer organiza cuatro componentes en un sistema coherente:

Componente	Función
Embedding posicional	Codifica palabras y su posición
Atención multi-cabezal	Relaciona todas las palabras en paralelo
Red feed-forward	Transforma cada posición de forma independiente
Normalización y residual	Estabiliza el entrenamiento

La arquitectura se propuso en Vaswani et al. (2017) para traducción automática y se extendió a prácticamente todas las tareas del NLP.

Vaswani et al. (2017). “Attention Is All You Need.” NeurIPS. https://arxiv.org/abs/1706.03762

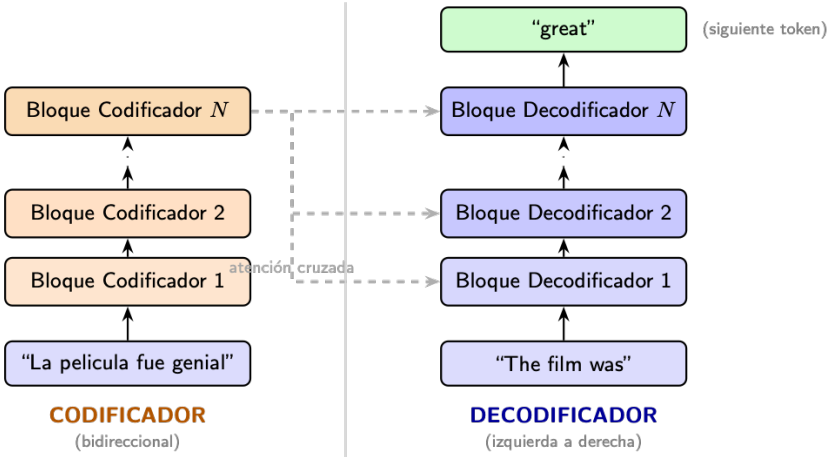
Visión General

El Transformer original contiene dos módulos principales: el **codificador** y el **decodificador**.

Codificador: Procesa la secuencia de entrada. Puede ver todas las palabras a la vez (bidireccional). Produce una representación contextualizada de cada posición.

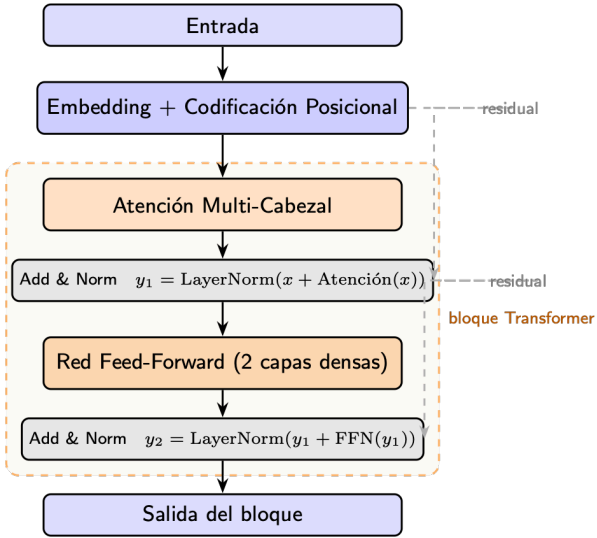
Decodificador: Genera la secuencia de salida de izquierda a derecha. Solo puede ver posiciones anteriores en la secuencia generada. Atiende también a la salida del codificador.

Cada módulo apila N bloques idénticos. El artículo original usa $N = 6$.



Goodfellow, Bengio & Courville (2016). Deep Learning, Cap. 12. MIT Press. `deeplearningbook.org`

Estructura Interna (Transformer)



Por qué son Necesarias

Conexión residual: La entrada de una sub-capa se suma a su salida.

$$y = \text{LayerNorm}(x + \text{Subcapa}(x))$$

Esto permite entrenar redes muy profundas sin degradación de gradientes. Sin estas conexiones, los modelos con 12 o más bloques no convergen de forma estable.

Normalización de capa (***Layer Normalization***): Normaliza cada posición de la secuencia de forma independiente. Es distinta a la normalización de batch: no depende del tamaño del lote.

*Ba et al. (2016). “Layer Normalization.” `arxiv.org/abs/1607.06450` ***He et al. (2016). “Deep Residual Learning for Image Recognition.” CVPR. `arxiv.org/abs/1512.03385`***

Diferencias Clave

Aspecto	Codificador	Decodificador
Dirección	Bidireccional	Solo izquierda a derecha
Tipo de atención	Auto-atención completa	Auto-atención enmascarada + atención cruzada
Objetivo	Comprender la entrada	Generar la salida
Modelos derivados	BERT, RoBERTa	GPT, LLaMA
Tareas típicas	Clasificación, NER	Traducción, generación

Para clasificación de textos, se usa solo el codificador.

Raffel et al. (2020). “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” JMLR. `arxiv.org/abs/1910.10683`

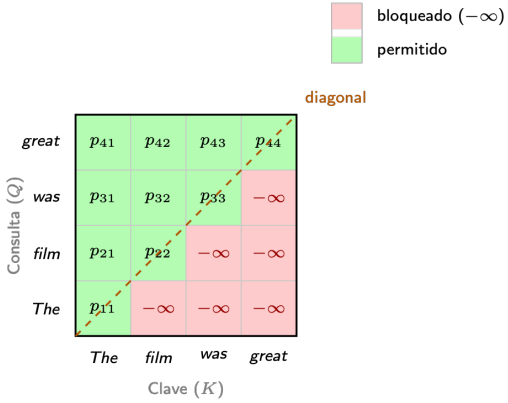
Por qué se Enmascara en el Decodificador

El codificador puede ver toda la secuencia de entrada, lo cual es deseable.

El decodificador **no puede ver el futuro** al generar: si está produciendo la palabra en la posición t, solo puede atender a las posiciones $1, 2, \ldots, t-1$.

Implementación: Los puntajes de atención de posiciones futuras se reemplazan por $-\infty$ antes del softmax. Tras el softmax, esos pesos son cero.

$$\text{puntaje}(i, j) = -\infty \quad \text{si } j > i$$



Sin esta máscara, el modelo aprendería a “copiar” la respuesta en lugar de generarla.

Vaswani et al. (2017), Sección 3.2.3 — describe la construcción de la máscara triangular inferior.

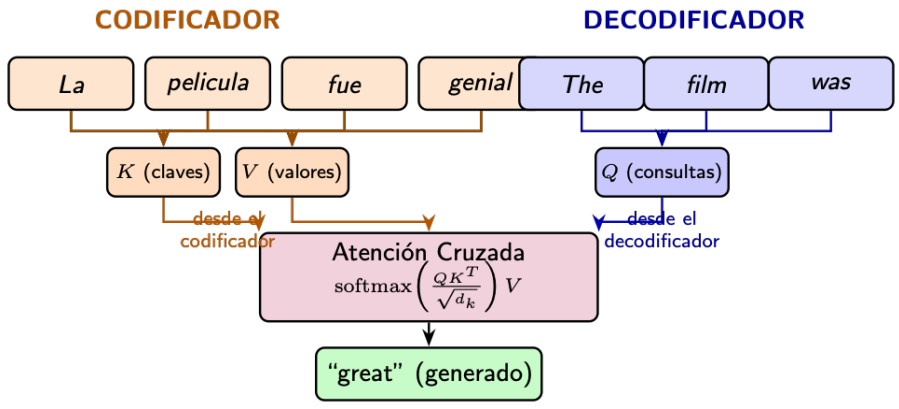
El Decodificador Atiende al Codificador

El decodificador tiene una segunda sub-capa de atención, llamada **atención cruzada** (***cross-attention***):

- Las **consultas** Q provienen del decodificador (lo que se está generando)
- Las **claves** K y **valores** V provienen del codificador (la entrada procesada)

Esto permite que cada posición generada decida a qué partes de la entrada original prestar atención.

Ejemplo: Al traducir “La película fue genial” al inglés, cuando el decodificador genera “film”, puede atender fuertemente a “película” en el codificador.



Bahdanau, Cho & Bengio (2015). “Neural Machine Translation by Jointly Learning to Align and Translate.” `arxiv.org/abs/1409.0473`

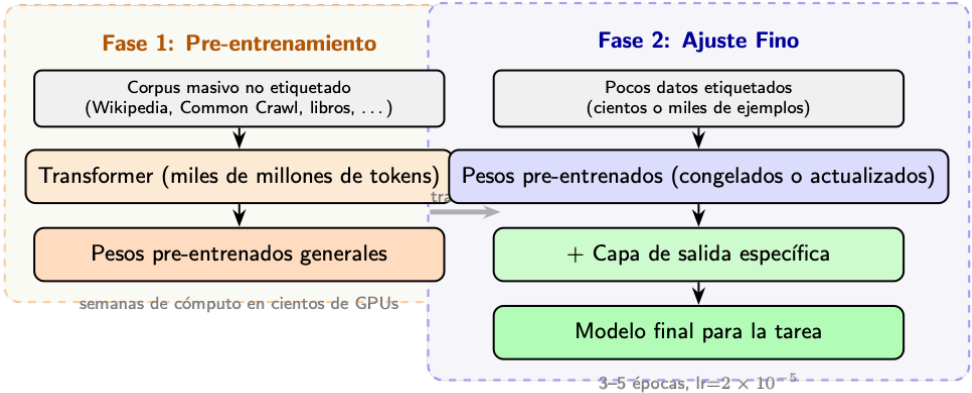
El Cambio de Paradigma

Antes del pre-entrenamiento, cada tarea requería un modelo entrenado desde cero con datos etiquetados. Esto era costoso y limitaba la generalización.

El nuevo paradigma tiene dos fases:

1. **Pre-entrenamiento**: Entrenar sobre un corpus masivo de texto no etiquetado. El modelo aprende la estructura general del lenguaje: gramática, semántica, hechos del mundo.
2. **Ajuste fino** (***fine-tuning***): Adaptar el modelo pre-entrenado a una tarea específica con pocos datos etiquetados.

El ajuste fino requiere solo 3–5 épocas con tasas de aprendizaje bajas (2×10^{-5}). Los pesos pre-entrenados son un excelente punto de partida.



Howard & Ruder (2018). “Universal Language Model Fine-Tuning for Text Classification.” ACL. `arxiv.org/abs/1801.06146`

Arquitectura y Pre-entrenamiento

BERT (***Bidirectional Encoder Representations from Transformers***) usa únicamente el módulo codificador del Transformer.

Configuración original:

Versión	Bloques	Cabezas	Parámetros
BERT-base	12	12	110 millones
BERT-large	24	16	340 millones

Entrenado sobre 3,300 millones de palabras (Wikipedia en inglés + BookCorpus).

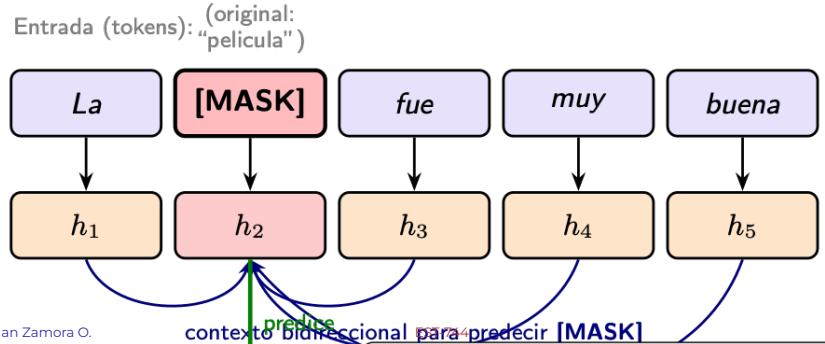
Devlin et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." NAACL. `arxiv.org/abs/1810.04805`

Cómo Aprende sin Etiquetas

BERT se pre-entrena con dos objetivos simultáneos:

Objetivo 1: Modelo de lenguaje enmascarado (***Masked Language Model***, MLM)

Se enmascara aleatoriamente el 15% de los tokens. El modelo debe predecir las palabras enmascaradas a partir del contexto bidireccional.



Objetivo 2: Predicción de siguiente oración (***Next Sentence Prediction***, NSP)

El modelo recibe dos oraciones y debe decidir si la segunda sigue a la primera en el texto original.

Devlin et al. (2019). Ibid. La tarea MLM es la contribución más importante; el NSP fue cuestionado posteriormente por Liu et al. (2019) en RoBERTa.

Arquitectura y Pre-entrenamiento

GPT (***Generative Pre-trained Transformer***) usa únicamente el módulo decodificador con atención enmascarada.

Objetivo de pre-entrenamiento: Predicción del siguiente token.

Dado el texto “La película fue muy”, el modelo predice “buena”. Luego, dado “La película fue muy buena”, predice el siguiente token. Esto es simplemente modelado del lenguaje clásico.

Versión	Parámetros	Año
GPT-1	117 millones	2018
GPT-2	1,500 millones	2019
GPT-3	175,000 millones	2020

Brown et al. (2020). “Language Models are Few-Shot Learners.” NeurIPS. `arxiv.org/abs/2005.14165`

Comparación Conceptual

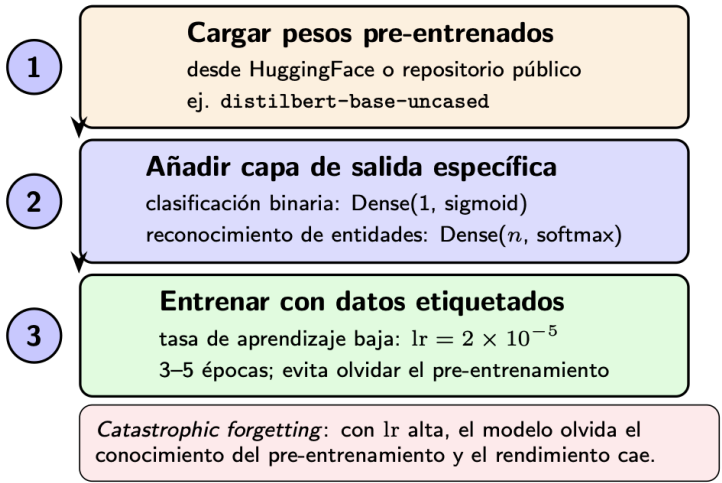
Aspecto	BERT	GPT
Módulo	Solo codificador	Solo decodificador
Dirección	Bidireccional	Solo izquierda a derecha
Pre-entrenamiento	Texto enmascarado (MLM)	Siguiente token
Fortaleza principal	Comprensión	Generación
Uso típico	Clasificación, extracción	Chatbots, resumen, traducción
Ajuste fino	Añadir capa de salida	Instrucciones en contexto

La elección depende de la tarea. Para clasificar textos, BERT es la opción natural. Para generar texto, GPT.

Rogers, Kovaleva & Rumshisky (2020). “A Primer in BERTology.” TACL. `arxiv.org/abs/2002.12327` — Revisión exhaustiva de qué aprende BERT.

Del Modelo General a la Tarea Específica

El ajuste fino consiste en tres pasos:



La Biblioteca Estándar

HuggingFace Transformers es la biblioteca de código abierto más utilizada para modelos de lenguaje pre-entrenados.

Funcionalidades principales:

- Más de 50,000 modelos pre-entrenados descargables
- Soporte para TensorFlow, PyTorch y JAX
- Descarga automática de pesos y tokenizadores
- ***Pipelines*** de alto nivel para tareas comunes
- Comunidad activa con nuevos modelos cada semana

El flujo de trabajo estándar con HuggingFace es: cargar el tokenizador, tokenizar los textos, cargar el modelo, compilar, ajustar.

Wolf et al. (2020). “Transformers: State-of-the-Art Natural Language Processing.” EMNLP. `arxiv.org/abs/1910.03771`

WordPiece: Subpalabras como Unidad

BERT no tokeniza por palabras completas. Usa el algoritmo **WordPiece**, que divide palabras poco frecuentes en subpalabras.

Ejemplo:

Texto	Tokens
`clasificación''` | [clasifica'', `\#\#ción''] |` | pre-entrenamiento''	[`pre''`,-'', `entrenamiento''] |` | película''

Ventajas: El vocabulario es pequeño (30,000 tokens). Palabras nuevas o en idiomas no vistos siempre tienen representación.

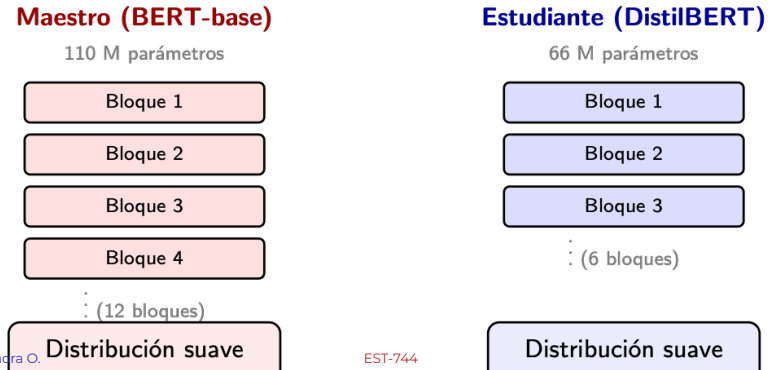
Token especial: BERT añade `[CLS]` al inicio de cada secuencia. La representación de ese token se usa para clasificación.

Schuster & Nakamura (2012). “Japanese and Korean Voice Search.” ICASSP. Origen del algoritmo WordPiece.

BERT más Liviano

Entrenar y usar BERT-base en producción puede ser lento o costoso. **DistilBERT** es una versión comprimida mediante destilación de conocimiento (***knowledge distillation***):

Un modelo pequeño (***estudiante***) es entrenado para imitar las probabilidades de salida de un modelo grande (***maestro***), no solo para predecir las etiquetas correctas.



Modelo	Parámetros	Velocidad relativa	Precisión relativa
BERT-base	110M	1x	100%
DistilBERT	66M	1.6x	97%
ALBERT	12M	5x	95%
RoBERTa	355M	0.7x	102%

Recomendación práctica: Comenzar con DistilBERT para experimentos rápidos; escalar a RoBERTa si se requiere máxima precisión.

Sanh et al. (2019). "DistilBERT, a distilled version of BERT." `arxiv.org/abs/1910.01108`

Flujo Completo con DistilBERT

El flujo para clasificación con DistilBERT tiene cuatro pasos:

Paso 1 — Tokenización: Convertir textos a identificadores de subpalabras, máscaras de atención y tokens de tipo. La longitud máxima habitual es 128 tokens.

Paso 2 — Construcción del dataset: Crear un objeto `tf.data.Dataset` con los encodings y etiquetas.

Paso 3 — Ajuste fino: Compilar con `Adam(lr=2e-5)` y `SparseCategoricalCrossentropy`. Entrenar durante 3 épocas.

Paso 4 — Predicción: Pasar el encoding por el modelo, obtener logits, aplicar softmax.

Sun et al. (2019). Ibid.

Qué Puede Hacer un Transformer

Tarea	Módulo	Descripción
Clasificación	Codificador	Añadir capa densa al token [CLS]
Reconocimiento de entidades	Codificador	Clasificar cada token individualmente
Respuesta a preguntas	Codificador	Encontrar el fragmento de respuesta en un párrafo
Traducción	Codificador-Decodificador	Transformar secuencia a secuencia
Resumen	Codificador-Decodificador	Condensar un documento
Generación de texto	Decodificador	Completar una secuencia dada

La arquitectura base es la misma; cambia la capa de salida y el tipo de datos de entrenamiento.

Por qué los Modelos Grandes Funcionan Mejor

Los Transformers exhiben **leyes de escala** (***scaling laws***): el rendimiento mejora de forma predecible al aumentar tres factores.

Factor	Ejemplo
Número de parámetros	BERT 110M → GPT-3 175,000M
Cantidad de datos de entrenamiento	Wikipedia → Common Crawl (1T tokens)
Cómputo de entrenamiento	Horas → Meses en miles de GPUs

Esta predictibilidad es inusual en aprendizaje profundo. En contraste, las RNN muestran rendimientos decrecientes al aumentar tamaño.

La consecuencia práctica es que invertir en modelos más grandes tiene retorno medible, lo que justifica el gasto computacional en la industria.

Kaplan et al. (2020). “Scaling Laws for Neural Language Models.” `arxiv.org/abs/2001.08361`

Por qué los Transformers Desplazaron a las RNN

Las RNN deben procesar tokens de forma secuencial. El token h_t depende de h_{t-1}, lo que impide paralelizar.

Los Transformers calculan la atención entre todos los pares de tokens simultáneamente:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Esta operación es una multiplicación de matrices, que las GPUs y TPUs ejecutan en paralelo de forma altamente eficiente.

Modelo	Cómputo por capa	Cómputo secuencial
RNN	$O(n \cdot d^2)$	$O(n)$ pasos
Transformer	$O(n^2 \cdot d)$	$O(1)$ pasos

Vaswani et al. (2017), Tabla 1. La comparación de complejidad está en la Sección 4.

Lo que los Transformers no Hacen Bien

Los Transformers son poderosos, pero tienen limitaciones concretas.

Cuadrático en longitud: La atención tiene complejidad $O(n^2)$ en la longitud de la secuencia. Secuencias de más de 512 tokens son costosas. Existen variantes eficientes (Longformer, BigBird) pero son más complejas.

Sin razonamiento explícito: Los Transformers asocian patrones estadísticos; no razonan en el sentido lógico. Fallan en tareas que requieren seguir reglas precisas.

Sensibles a distribución: El rendimiento cae significativamente cuando los datos de prueba difieren de los de entrenamiento.

Sesgos del pre-entrenamiento: Los modelos absorben los sesgos presentes en el corpus de entrenamiento (sesgos de género, culturales, etc.).

Bender et al. (2021). "On the Dangers of Stochastic Parrots." FAccT. Discusión sobre riesgos éticos de modelos grandes.

Precauciones para Uso en Producción

Los modelos pre-entrenados aprenden desde texto escrito por humanos. Ese texto contiene sesgos históricos y culturales que el modelo reproduce y puede amplificar.

Sesgos frecuentes documentados:

- Asociaciones de género con profesiones
- Estereotipos raciales y culturales
- Representación desigual de idiomas minoritarios

Prácticas recomendadas antes de desplegar un modelo:

- Auditar predicciones en subgrupos demográficos
- Medir métricas de equidad (***fairness***)
- Documentar los datos de entrenamiento
- Establecer mecanismos de reporte de errores

Blodgett et al. (2020). “Language (Technology) is Power: A Critical Survey of Bias in NLP.” ACL. `arxiv.org/abs/2005.14050`

Extensiones Recientes

La combinación de atención, pre-entrenamiento y escalado ha demostrado funcionar en dominios más allá del NLP.

Dominio	Modelo Representativo	Tarea
Visión computacional	ViT (2021)	Clasificación de imágenes
Audio	Wav2Vec 2.0 (2020)	Reconocimiento de voz
Multimodal	CLIP (2021)	Alineación imagen-texto
Proteínas	AlphaFold 2 (2021)	Predicción de estructura 3D
Grafos	Graph Transformer	Predicción de propiedades moleculares

El mensaje central es que la arquitectura Transformer es un componente general para modelar relaciones entre elementos de una secuencia, independiente del dominio.

Dosovitskiy et al. (2021). “An Image is Worth 16x16 Words.” ICLR. `arxiv.org/abs/2010.11929`

Resumen de esta parte del curso

Clase	Tema	Idea Central
1	Redes Neuronales	Aprendizaje por gradiente
2	Embeddings	Palabras como vectores
3	RNN / LSTM / CNN	Procesamiento de secuencias
4	Atención	Contexto paralelo
5	Transformers	Pre-entrenamiento escalable

Trayectoria: Las arquitecturas mejoraron su capacidad expresiva. El entrenamiento se aceleró mediante paralelización. Los modelos crecieron para aprovechar datos masivos no etiquetados.

Para Profundizar

- **Vaswani et al. (2017).** “Attention Is All You Need.” ***NeurIPS***. `arxiv.org/abs/1706.03762` — El artículo fundacional; presenta la arquitectura completa del Transformer.
- **Devlin et al. (2019).** “BERT: Pre-training of Deep Bidirectional Transformers.” ***NAACL***. `arxiv.org/abs/1810.04805` — Introducción de BERT y el paradigma de pre-entrenamiento bidireccional.
- **Brown et al. (2020).** “Language Models are Few-Shot Learners.” ***NeurIPS***. `arxiv.org/abs/2005.14165` — GPT-3 y aprendizaje en contexto (***in-context learning***).
- **Sanh et al. (2019).** “DistilBERT.” ***arXiv***. `arxiv.org/abs/1910.01108` — Destilación de conocimiento aplicada a BERT.
- **Kaplan et al. (2020).** “Scaling Laws for Neural Language Models.” ***arXiv***. `arxiv.org/abs/2001.08361` — Marco cuantitativo de cómo escala el rendimiento.

Para Trabajar de Forma Independiente

1. Ajusta DistilBERT sobre un dataset de tu elección. Compara con el modelo LSTM de la Clase 3. Registra AUROC y F1.
2. Compara BERT vs. DistilBERT en la misma tarea. Mide: tiempo de entrenamiento, parámetros, precisión final.
3. Experimento de pocos datos (***few-shot***): Ajusta con 10, 50 y 100 ejemplos etiquetados. ¿Cuántos datos son suficientes?
4. Intenta ajustar sobre datos de un dominio distinto al del pre-entrenamiento. Observa la caída de rendimiento.
5. Visualiza la atención de BERT sobre una oración. Identifica patrones en distintas cabezas.